

Optimizers & Regularization Crash Course

Momentum, Adam, dropout, weight decay, and batch norm — what they are, why they exist, when to use them

1. Why This Matters

Plain gradient descent (" $W = W - \text{learning_rate} \times dW$ ", applied identically every step) works, but has two real limitations in practice: it can be slow or unstable to converge on complex loss landscapes, and it has no built-in defense against overfitting. Optimizer variants address the first problem; regularization techniques address the second.

2. Optimizer Variants

Momentum

Plain SGD only looks at the current gradient, which makes it prone to zig-zagging when the loss landscape is much steeper in one direction than another. Momentum fixes this by accumulating a running average of past gradients — like a ball rolling downhill that builds up speed in a consistent direction and doesn't instantly change course just because one step's gradient was noisy.

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

A real characteristic worth knowing: momentum's built-up velocity can cause it to overshoot past the minimum before settling — this isn't a bug, it's an inherent tradeoff of the technique, usually still worth it for the smoother overall path.

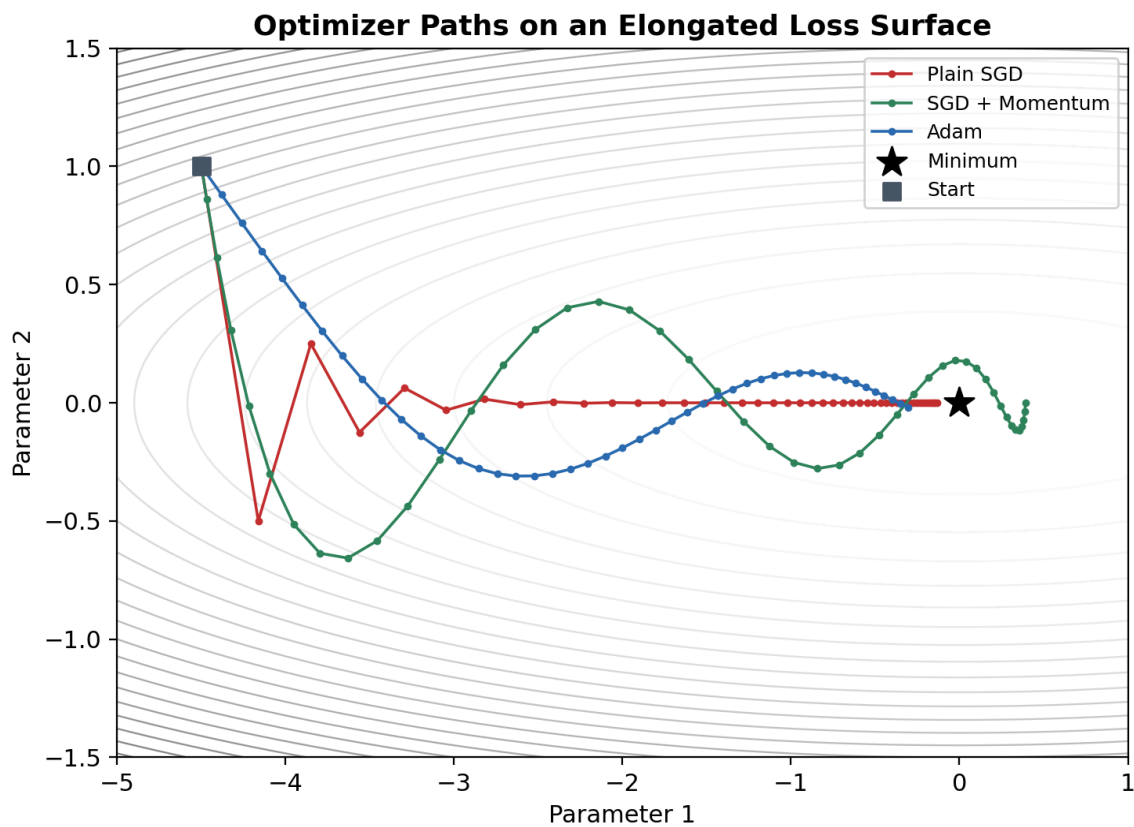
Adam

Adam (Adaptive Moment Estimation) goes further than momentum: it tracks a running average of both the gradient AND the squared gradient, using the second to adapt the effective learning rate for EACH parameter individually. Parameters getting large, consistent gradients get smaller effective steps; parameters with small or noisy gradients get relatively larger ones.

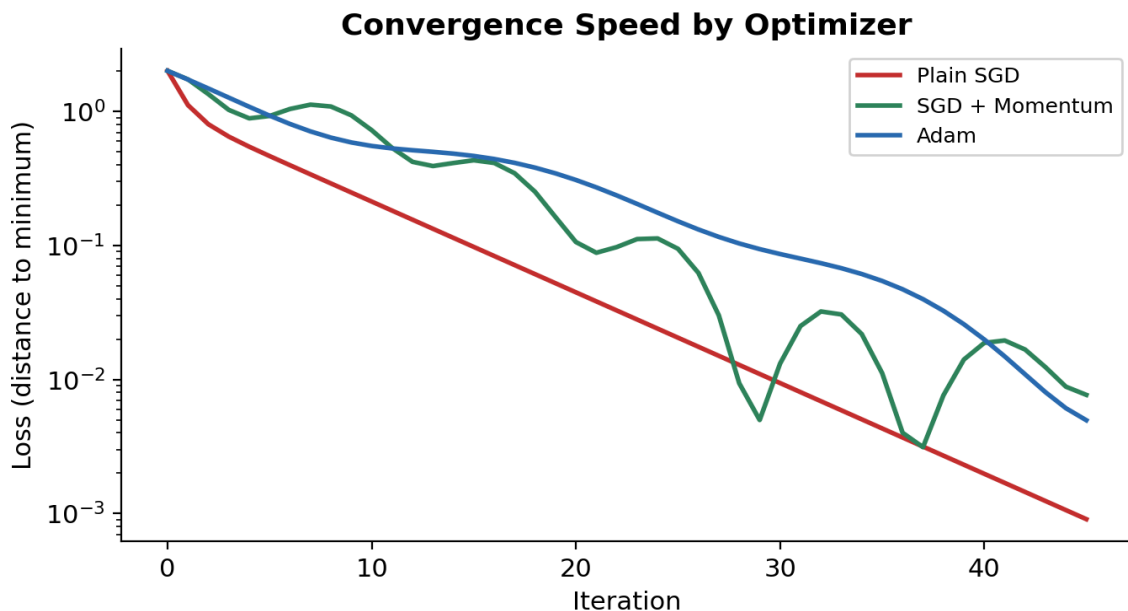
```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

This adaptivity is why Adam is the most common default optimizer in modern deep learning — it typically converges reliably with less manual learning-rate tuning than plain SGD. Note Adam's typical learning rate (0.001) is usually smaller than SGD's, since its adaptive scaling already does some of the work a larger SGD rate would otherwise need to.

Visualizing the difference



Paths from the same starting point on a loss surface that's steep in one direction and shallow in the other. SGD zig-zags along the steep direction; momentum smooths this but can overshoot past the minimum from built-up velocity; Adam adapts its step size per-direction and converges most directly.



Distance to the minimum over time, for the same three runs shown above.

An honest note on this figure: on this simple, smooth, 2-parameter toy problem, plain SGD with a well-chosen learning rate actually converges cleanly — sometimes just as fast as the adaptive methods. That's a real and useful thing to notice, not a flaw in the example. Momentum and Adam's real advantages show up on ACTUAL neural network loss landscapes: thousands to millions of parameters, noisy mini-batch gradients (not the smooth, deterministic gradient used here), saddle points, and regions of very different curvature across many directions at once — conditions a 2-parameter toy bowl can't fully capture. The qualitative behavior (zig-zag vs. overshoot vs. adapt) is the real lesson here, not a declared "winner" from this specific figure.

Quick reference

Optimizer	How it works	Use case
Plain SGD	Steps directly opposite the current gradient, every time, with no memory of past steps.	Simple problems, or when you want full manual control over the learning rate schedule.
SGD + Momentum	Accumulates a running average of past gradients, smoothing out noisy directions and building speed in consistent ones.	A common upgrade over plain SGD when training is slow or oscillating; still needs manual learning rate tuning.
Adam	Adapts the effective learning rate per-parameter, using running averages of both the gradient and its square.	The default choice for most modern deep learning — fast convergence with less manual tuning.

3. Regularization

Regularization techniques exist to fight overfitting — a model that performs great on training data but poorly on new data because it memorized specifics rather than learning generalizable patterns.

L2 Weight Decay

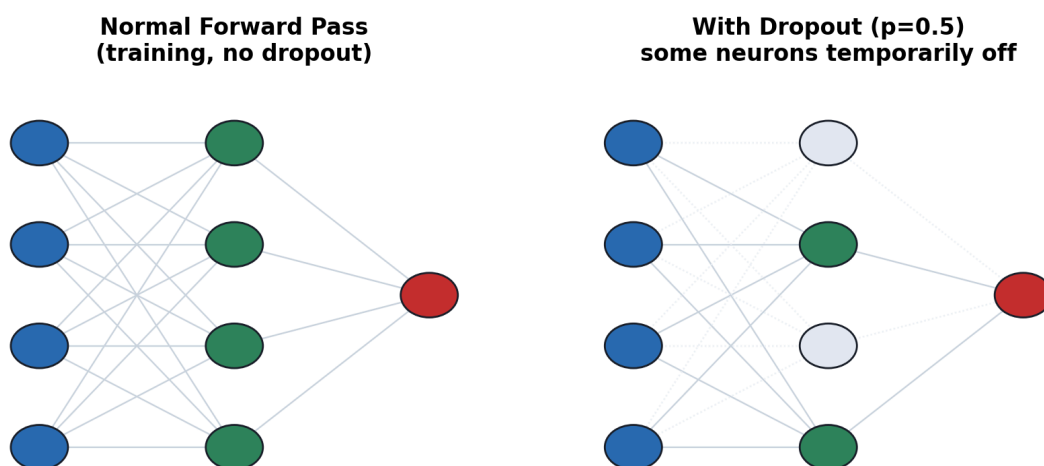
Adds a penalty to the loss proportional to the size of the weights themselves, discouraging any single weight from growing very large. Large weights are often a sign of the model fitting noise or specific training examples rather than genuine patterns.

```
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
```

Dropout

During training, randomly "turns off" (zeroes out) a fraction of neurons on each forward pass. This prevents the network from becoming overly reliant on any single neuron, forcing it to learn more redundant, robust representations. Dropout is automatically disabled during evaluation.

```
self.dropout = nn.Dropout(p=0.5) # in __init__  
x = self.dropout(x) # in forward(), typically before the final layer
```



Left: a normal forward pass, every neuron active. Right: with dropout, a random subset of hidden neurons (and their connections) are temporarily zeroed out for that pass — a different random subset each time.

Batch Normalization

Normalizes the activations within each mini-batch to have consistent mean/variance before passing to the next layer — conceptually related to why you scale input features before training, but applied to intermediate activations throughout the network, not just the raw input. This tends to stabilize and speed up training, especially in deeper networks.

```
self.bn1 = nn.BatchNorm2d(16)    # after a conv layer with 16 output channels
x = self.bn1(self.conv1(x))      # applied before the activation function
```

Quick reference

Technique	What it does	Use case
L2 weight decay	Penalizes large weights during training, encouraging smaller, more evenly-distributed values.	General-purpose overfitting prevention; cheap to add via the optimizer's <code>weight_decay</code> argument.
Dropout	Randomly zeroes out a fraction of neurons on each training forward pass.	Large fully-connected layers prone to overfitting; very common in CNN/Transformer architectures.
Batch normalization	Normalizes activations within each mini-batch before the next layer.	Deeper networks where training stability and speed are a concern.

4. A Caveat Worth Remembering

Regularization only helps when overfitting is actually a real risk — typically when a model is large/complex relative to how much training data is available. On a large, relatively easy dataset, adding dropout or weight decay might barely change (or even slightly hurt) validation accuracy, and that's not a failure of the technique — it's a sign there wasn't much overfitting to fix in the first place. Always compare against an unregularized baseline before assuming regularization helped.

5. Where This Connects to Your Roadmap

Your Part 3 notebook trains the same CNN architecture three times, varying only the optimizer, so you can watch the qualitative differences in Section 2's diagrams play out on your own real training curves rather than a toy example. The regularization section then extends your CNN with dropout (and optionally batch norm) and asks you to honestly assess whether it helped — tying directly into Section 4's caveat above.